

React Interview Prep

Topic 4 — State: `useState`, Re-renders & Mental Model

BEGINNER → INTERMEDIATE → EXPERT

■ Topic	State — <code>useState</code> , Re-renders & Mental Model
■ Covers	<code>useState</code> · Object/Array state · Re-render cycle · Batching · Lifting state
■ Series	React Interview Prep — Session 4

by Akshay Chavhan • React Interview Series

1. What is State?

State is a component's **internal memory** — data that can change over time and triggers a UI update when it does. Unlike props (passed from parent), state is **owned and managed by the component itself**.

■ *State = component memory. Change it → React re-renders the UI automatically.*

2. useState Hook

Syntax

```
const [count, setCount] = useState(0);  
// ↑ ↑ ↑  
// current updater fn initial value (used ONCE on first render)
```

Full Example

```
function Counter() {  
  const [count, setCount] = useState(0);  
  return (  
    <div>  
      <p>Count: {count}</p>  
      <button onClick={() => setCount(count + 1)}>+</button>  
      <button onClick={() => setCount(count - 1)}>-</button>  
    </div>  
  );  
}
```

3. State with Objects & Arrays

Objects

```
// ■ WRONG – mutates directly  
user.name = 'React';  
  
// ■ CORRECT – spread + new obj  
setUser({ ...user, name: 'React' });
```

Arrays

```
// ■ Add  
setItems([...items, 'Mango']);  
  
// ■ Remove  
setItems(items.filter(i => i !== 'Banana'));  
  
// ■ WRONG  
items.push('Mango'); // no re-render!
```

■ *Never mutate state directly. React uses reference equality — same ref = no re-render detected.*

4. How Re-renders Work



5. Expert Patterns

Functional Updates — Avoid Stale State

```
// ■ Stale – all 3 read same snapshot → only +1 total
setCount(count + 1); setCount(count + 1); setCount(count + 1);

// ■ Functional – always gets latest state → +3 total
setCount(prev => prev + 1);
setCount(prev => prev + 1);
setCount(prev => prev + 1);
```

■ *Use functional updates whenever new state depends on previous state.*

State Batching (React 18)

```
function handleClick() {
  setCount(c => c + 1); // not rendered yet
  setName('Akshay'); // not rendered yet
  setOnline(true); // not rendered yet
  // React 18: ALL three batched → ONE re-render ■
}
```

■ *React 18 automatic batching extends to async callbacks (setTimeout, fetch) — not just event handlers.*

Lazy Initializer

```
// ■ expensiveCalc() runs on EVERY render
const [val, setVal] = useState(expensiveCalc());

// ■ Pass a function – runs ONLY on first render
const [val, setVal] = useState(() => expensiveCalc());
```

Lifting State Up

```
function Parent() {
  const [count, setCount] = useState(0); // shared state lives here
  return (
    <>
    <Display count={count} /> { /* reads state */ }
    <Controls setCount={setCount} /> { /* updates state */ }
    </>
  );
}
```

■ *Lifting state up = move state to nearest common ancestor of components that need it.*

6. Props vs State — Quick Reference

	Props	State
--	-------	-------

Owned by	Parent component	Component itself
Mutable?	No — read only	Yes — via setter function
Triggers re-render	Yes (when parent updates)	Yes (via setState)
Set by	Parent passes down	useState / useReducer
Purpose	Pass data into component	Track changing internal data

7. Interview Q&A;

Q

Beginner

Q1. What is state in React?

→ State is a component's internal memory — data that can change over time and triggers a re-render when updated. Unlike props which come from the parent, state is owned and managed by the component itself.

Q

Beginner

Q2. What happens when you call a state setter function?

→ React schedules a re-render. The component function runs again top to bottom, produces new JSX, React diffs it against the previous Virtual DOM, and patches only the changed nodes in the real DOM.

Q

Intermediate

Q3. Why should you never mutate state directly?

→ React uses reference equality to detect changes. If you mutate the object directly, the reference stays the same, React sees no change, and won't re-render. Always pass a new value or new object/array to the setter.

Q

Intermediate

Q4. What is a stale state bug and how do you fix it?

→ It happens when you call the setter multiple times using the current state value — all calls read the same stale snapshot. Fix it with functional updates: `setCount(prev => prev + 1)` which always receives the latest state.

Q

Intermediate

Q5. What is state batching in React 18?

→ Batching groups multiple `setState` calls into one re-render. React 18 introduced automatic batching — even async operations like `setTimeout` and `fetch` callbacks are now batched, not just synchronous event handlers.

Q**Expert****Q6. What is lifting state up?**

→ When two sibling components need to share state, you move that state to their nearest common ancestor and pass it down as props. This is called lifting state up — it's React's primary pattern for sharing state between components.

Q**Expert****Q7. What is a lazy initializer in `useState`?**

→ Passing a function to `useState` instead of a value: `useState(() => expensiveCalc())`. The function only runs on the first render, not on every re-render. Use it when the initial state requires an expensive computation.